

Software Requirements Specification

for

Distributed Information System for Fitness Center Management

Version:	1.0 approved
Date:	March 15, 2026
Status:	Approved
Prepared by:	Development Team
Organization:	—

Copyright (c) 1999 by Karl E. Wieggers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History

1. Introduction

1.1 Purpose

1.2 Document Conventions

1.3 Intended Audience and Reading Suggestions

1.4 Product Scope

1.5 References

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 User Documentation

2.7 Assumptions and Dependencies

3. External Interface Requirements

3.1 User Interfaces

3.2 Hardware Interfaces

3.3 Software Interfaces

3.4 Communications Interfaces

4. System Features (50 features)

4.1 User Login

4.2 Password Reset via Email

4.3 JWT Token Refresh

4.4 Role-Based Access Control

4.5 Account Lockout

4.6 Automatic Session Expiry

4.7 Create Member Profile

4.8 Edit Member Profile

4.9 View Member Profile

4.10 Deactivate Member Account

4.11 Search Members

4.12 Filter Members by Status

- 4.13 Export Member List to CSV
- 4.14 Restricted Members List
- 4.15 Create Subscription
- 4.16 View Subscription Details
- 4.17 Renew Subscription
- 4.18 Cancel Subscription
- 4.19 Personalized Subscription Pricing
- 4.20 Automatic Subscription Expiry
- 4.21 Subscription Type Configuration
- 4.22 Online Payment via Gateway
- 4.23 View Payment History
- 4.24 Payment Receipt Generation
- 4.25 Monthly Revenue Report
- 4.26 Refund Request Tracking
- 4.27 Create Training Class
- 4.28 Edit Training Class Details
- 4.29 Cancel Training Class
- 4.30 View Class Schedule
- 4.31 Class Capacity Enforcement
- 4.32 Assign Trainer to Class
- 4.33 Make a Class Reservation
- 4.34 Cancel a Reservation
- 4.35 View Own Reservations
- 4.36 View All Reservations for a Class
- 4.37 Waitlist Management
- 4.38 Reservation Confirmation Notification
- 4.39 Reservation Reminder Notification
- 4.40 Create Trainer Profile
- 4.41 Edit Trainer Profile
- 4.42 Deactivate Trainer Account
- 4.43 View Trainer Schedule
- 4.44 Trainer Availability Management
- 4.45 Create Facility
- 4.46 Edit Facility Details

- 4.47 Facility Occupancy Report
- 4.48 Facility Availability Calendar
- 4.49 Member Activity Report
- 4.50 Subscription Expiry Notifications
- 5. Other Nonfunctional Requirements
 - 5.1 Performance Requirements
 - 5.2 Safety Requirements
 - 5.3 Security Requirements
 - 5.4 Software Quality Attributes
 - 5.5 Business Rules
- 6. Other Requirements
- 7. Technical Architecture
- Appendix A: Glossary
- Appendix B: Analysis Models
- Appendix C: To Be Determined List

Revision History

Name	Date	Reason For Changes	Version
Dev Team	March 15, 2026	Initial version	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the **Distributed Information System for Fitness Center Management**, version 1.0. It covers the entire system including modules for authentication, member management, subscriptions, reservations, payments, trainer and facility management, and reporting. This SRS is the authoritative reference for all implementation and testing activities.

1.2 Document Conventions

Requirements are uniquely identified with the prefix **REQ-** followed by a sequential number. Priorities are indicated inline per feature as: **High (H)**, **Medium (M)**, or **Low (L)**. Higher-level requirements do not automatically inherit priority to sub-requirements; each REQ carries its own implied priority from its parent feature section. The term *user* refers generically to any authenticated person interacting with the system. The term *system* refers to the complete software platform including backend, frontend, and database.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- **Developers:** Read sections 1, 2, 3, 4, and 7 in full. Section 4 contains all implementable requirements.
- **Testers and QA:** Focus on Section 4 (functional requirements REQ-N) and Section 5 (nonfunctional). Each REQ is a testable acceptance criterion.
- **Project Managers:** Sections 1, 2, and 5 provide scope and constraints. Section 4 priorities guide sprint planning.
- **End Client / Stakeholders:** Sections 1, 2, and 4 describe what the system does in business terms.

It is recommended to begin with sections 1 and 2 for context, then proceed to section 4 for feature details, followed by sections 5, 6, and 7 for constraints and architecture.

1.4 Product Scope

The Distributed Information System for Fitness Center Management is a web-based platform enabling comprehensive management of a fitness and wellness facility. Its primary objectives are: (1) automating administrative workflows for staff, (2) providing members with self-service access to subscriptions, reservations, and payment history, and (3) giving management actionable reporting on revenue and facility utilization. The system does not include point-of-sale hardware integration, wearable device connectivity, or dietary/nutrition tracking in this version.

1.5 References

- IEEE Std 830-1998 -- Recommended Practice for Software Requirements Specifications
- Project Vision and Scope Document, v1.0
- Internal UI/UX Style Guide
- GDPR -- Regulation (EU) 2016/679 of the European Parliament
- PCI-DSS v4.0 -- Payment Card Industry Data Security Standard

- WCAG 2.1 -- Web Content Accessibility Guidelines

2. Overall Description

2.1 Product Perspective

This is a new, self-contained product and not a component of a larger existing system. It is composed of four main components: (1) a React single-page application served to the client browser, (2) a FastAPI backend exposing a versioned REST API, (3) a PostgreSQL relational database for persistent storage, and (4) third-party integrations for payments and email/SMS notifications. All user access occurs exclusively through the web interface. The system requires internet connectivity and a modern web browser.

2.2 Product Functions

At a high level, the system must perform the following major functions:

- Authenticate users and enforce role-based access (Administrator, Member, Trainer)
- Manage member profiles including personal data and subscription status
- Create, renew, and cancel subscriptions with both fixed and personalized pricing
- Process online payments and maintain a complete payment history
- Schedule and manage group training classes with capacity limits
- Allow members to reserve and cancel spots in training classes
- Manage trainer profiles, schedules, and availability
- Manage training facility definitions and availability
- Generate revenue, occupancy, and member activity reports
- Send automated email and SMS notifications for reservations and subscription events

2.3 User Classes and Characteristics

The system distinguishes three user classes with different privileges and usage patterns:

User Class	Characteristics	Privileges
Administrator	Technical or administrative staff. Full system familiarity expected. High-frequency daily use.	Full CRUD on all entities. Access to all reports. Manage subscription types and pricing. Manage facilities and equipment.
Member	Fitness center client. Varying technical background. Primarily accesses own data.	View and update own profile. View active subscription and payment history. Make and cancel class reservations.
Trainer	Center staff member. Basic IT knowledge assumed. Accesses system to manage schedule.	View and manage own schedule. View reservations for own classes. Update own availability.

2.4 Operating Environment

The backend application server will run on a Linux-based server (Ubuntu 22.04 LTS or equivalent) using Python 3.11+ with the Uvicorn ASGI server. The database will be PostgreSQL 15+ on a dedicated server instance. The frontend will be served as static assets from a CDN or web server. The system must be accessible from any modern browser (Chrome 120+, Firefox 120+, Safari 17+,

Edge 120+) on both desktop and mobile devices. No native mobile application is required in version 1.0.

2.5 Design and Implementation Constraints

- All personal data handling must comply with GDPR (EU 2016/679). A data processing agreement must be in place with any third-party service.
- Online payment processing must go through a PCI-DSS certified gateway. The system must never store raw card data.
- All client-server communication must use HTTPS with TLS 1.2 or higher.
- The backend API must be versioned under /api/v1 to allow future non-breaking evolution.
- The system must be containerizable (Docker) to support reproducible deployments.
- Database schema changes must be managed through versioned Alembic migrations; direct DDL changes are prohibited.

2.6 User Documentation

The following user documentation components will be delivered alongside the software:

- **Online User Manual (HTML):** Full documentation accessible within the app, covering all features per user role.
- **Quick Start Guide (PDF):** A concise 2-4 page guide for new members covering login, subscription viewing, and reservations.
- **Administrator Guide (PDF):** Covering member management, subscription configuration, reporting, and system settings.
- **Contextual Help:** Tooltip-level help text on all non-obvious form fields and actions within the application.

2.7 Assumptions and Dependencies

- A functional third-party payment gateway (Stripe or PayU) account is assumed to be provisioned before go-live.
- An SMTP service or transactional email provider (e.g., SendGrid, AWS SES) is assumed to be available for sending notifications.
- An SMS gateway (e.g., Twilio) is assumed to be available for SMS reminders, though SMS is an optional channel.
- All end users are assumed to have access to a modern browser and a stable internet connection.
- The fitness center is assumed to operate in a single timezone. Multi-timezone support is out of scope for v1.0.
- Currency is fixed to RON (Romanian Leu) in v1.0. Multi-currency support is deferred.

3. External Interface Requirements

3.1 User Interfaces

The web interface will be a responsive single-page application built in React. It must comply with WCAG 2.1 Level AA accessibility standards. Every screen must include: a persistent navigation sidebar or top bar showing the current user's name and role, a breadcrumb trail for navigational context, inline validation messages on all form fields, a confirmation modal for all destructive actions (deletions, cancellations), and a consistent toast notification system for success and error states. The interface must be fully usable on screens as small as 375px wide (iPhone SE). Keyboard navigation must be fully functional for all interactive elements.

3.2 Hardware Interfaces

The system requires no special hardware on the client side beyond a device capable of running a modern web browser. The server infrastructure requires: a minimum of 4 vCPUs and 8 GB RAM for the application server, SSD-backed storage with a minimum of 100 GB for the database server, and a network connection with at least 100 Mbps bandwidth. No physical device interfaces (card readers, printers, turnstiles) are in scope for v1.0.

3.3 Software Interfaces

The system integrates with the following external software components:

- **Payment Gateway (Stripe or PayU):** The backend communicates with the gateway's REST API using a server-side secret key. The frontend uses the gateway's JavaScript SDK for secure card data collection. Webhooks from the gateway will update payment status in the database.
- **Email Provider (SendGrid / AWS SES):** The backend sends transactional emails (confirmations, reminders, receipts) via the provider's SMTP or HTTP API.
- **SMS Gateway (Twilio, optional):** The backend sends SMS reminders via the provider's REST API when a member has a phone number on file and has opted in.
- **PostgreSQL 15+:** The backend communicates with the database via SQLAlchemy 2.0 using asyncpg as the async driver.

3.4 Communications Interfaces

All client-to-server communication uses HTTPS over TLS 1.2 or higher. The API follows REST conventions: resources are nouns, HTTP verbs (GET, POST, PUT, DELETE) express actions, and responses use standard HTTP status codes. Request and response bodies are JSON. Authentication uses Bearer tokens (JWT) in the Authorization header. Webhook endpoints from the payment gateway accept POST requests and verify signatures using HMAC-SHA256. Email delivery uses SMTP over port 587 with STARTTLS. No custom binary protocols or persistent connections (WebSockets) are required in v1.0.

4. System Features

This section defines all 50 system features. Each feature follows the IEEE 830 structure: description and priority, stimulus/response sequences, and functional requirements (REQ-N). Features are organized into eight functional groups.

Authentication and Security (Features 4.1 - 4.6)

4.1 User Login

4.1.1 Description and Priority

Priority: High (H)

The system must allow users to authenticate using their registered email address and password. Upon successful authentication the system issues a signed JWT access token and a refresh token. The access token encodes the user's role and is required for all protected API endpoints.

4.1.2 Stimulus/Response Sequences

- User navigates to the login page -> System displays the login form with email and password fields.
- User submits valid credentials -> System validates, issues JWT access token and refresh token, redirects to the role-appropriate dashboard.
- User submits invalid credentials -> System displays a generic error message without specifying which field is wrong.

4.1.3 Functional Requirements

REQ-1: The login form must accept an email address and a password.

REQ-2: The system must validate the email/password combination against the stored bcrypt hash.

REQ-3: On success the system must return an access token (15-minute expiry) and a refresh token (7-day expiry) in the response body.

REQ-4: The system must redirect the user to the dashboard appropriate for their role (Administrator, Member, or Trainer).

4.2 Password Reset via Email

4.2.1 Description and Priority

Priority: High (H)

Users who have forgotten their password must be able to request a reset link sent to their registered email address. The reset link must be time-limited and single-use.

4.2.2 Stimulus/Response Sequences

- User clicks 'Forgot password' -> System displays an email input form.
- User submits a registered email -> System sends a reset link and displays a generic confirmation message.
- User clicks the reset link within 1 hour -> System displays the new password form.
- User submits new password -> System updates the hash, invalidates the link, and redirects to login.

4.2.3 Functional Requirements

REQ-5: The system must send a password reset email within 30 seconds of the request.

REQ-6: The reset token must expire after 60 minutes from issuance.

REQ-7: Each reset token must be single-use; clicking it a second time must return an error.

REQ-8: The new password must meet minimum complexity: at least 8 characters, one uppercase, one digit.

4.3 JWT Token Refresh

4.3.1 Description and Priority

Priority: High (H)

To maintain a seamless session without requiring frequent logins, the frontend must be able to silently exchange a valid refresh token for a new access token.

4.3.2 Stimulus/Response Sequences

- Access token expires -> Frontend automatically sends the refresh token to POST /auth/refresh.
- Refresh token is valid -> System issues a new access token; user's session continues uninterrupted.
- Refresh token is expired or revoked -> System returns 401; frontend redirects to login.

4.3.3 Functional Requirements

REQ-9: The POST /auth/refresh endpoint must accept a valid refresh token and return a new access token.

REQ-10: The system must reject refresh tokens that have been revoked (e.g., after a password change).

REQ-11: Refresh tokens must be stored securely and rotated on each use.

4.4 Role-Based Access Control

4.4.1 Description and Priority

Priority: High (H)

Every API endpoint and UI section must enforce access based on the authenticated user's role. The three roles are Administrator, Member, and Trainer, each with a distinct permission set.

4.4.2 Stimulus/Response Sequences

- Authenticated user requests a resource -> System checks the role claim in the JWT before processing.
- User role does not have permission -> System returns HTTP 403 Forbidden.
- User attempts to access another user's private data -> System returns HTTP 403 Forbidden.

4.4.3 Functional Requirements

REQ-12: Access control must be enforced server-side on every protected endpoint; client-side hiding is supplementary only.

REQ-13: Administrators must have access to all resources across all users.

REQ-14: Members must only access their own profile, subscription, payment history, and reservations.

REQ-15: Trainers must only access their own profile, schedule, and the reservation lists of their own classes.

4.5 Account Lockout After Failed Login Attempts

4.5.1 Description and Priority

Priority: High (H)

To protect against brute-force attacks, the system must temporarily lock an account after a configurable number of consecutive failed login attempts.

4.5.2 Stimulus/Response Sequences

- User submits incorrect credentials -> System increments the failed attempt counter for that email.
- Failed attempts reach the threshold (5) -> System locks the account and sends a notification email.
- Locked user attempts to log in -> System rejects the attempt with a clear lockout message.
- Lockout period (15 minutes) expires -> System automatically unlocks the account.

4.5.3 Functional Requirements

REQ-16: The system must lock an account after 5 consecutive failed login attempts.

REQ-17: The lockout duration must be 15 minutes.

REQ-18: The system must send a notification email to the account owner when a lockout is triggered.

REQ-19: Administrators must be able to manually unlock any account from the admin panel.

4.6 Automatic Session Expiry

4.6.1 Description and Priority

Priority: Medium (M)

To protect unattended sessions, the system must automatically invalidate a user's frontend session after a period of inactivity, requiring re-authentication.

4.6.2 Stimulus/Response Sequences

- User is inactive for 30 minutes -> Frontend detects inactivity and shows a warning modal.
- User ignores the warning for 60 seconds -> Frontend clears tokens and redirects to the login page.
- User interacts before timeout -> Inactivity timer is reset.

4.6.3 Functional Requirements

REQ-20: The inactivity timeout must be 30 minutes, configurable by an Administrator.

REQ-21: The system must display a warning dialog 60 seconds before the session expires.

REQ-22: On expiry, all locally stored tokens must be cleared and the user must be redirected to the login page.

Member Management (Features 4.7 - 4.14)

4.7 Create Member Profile

4.7.1 Description and Priority

Priority: High (H)

Administrators must be able to register new members in the system, creating both a user account and the associated member profile record.

4.7.2 Stimulus/Response Sequences

- Administrator navigates to Members > Add New -> System displays the member creation form.
- Administrator submits valid data -> System creates user account and member profile, displays success message.
- Administrator submits a duplicate email -> System displays a validation error without creating the record.

4.7.3 Functional Requirements

REQ-23: The creation form must capture: first name, last name, email address, phone number, and date of birth.

REQ-24: The system must validate that the email address is unique before creating the record.

REQ-25: The system must send a welcome email to the new member with a link to set their password.

REQ-26: The newly created member must have an 'inactive' subscription status by default.

4.8 Edit Member Profile

4.8.1 Description and Priority

Priority: High (H)

Administrators must be able to update any member's profile data. Members must be able to update their own profile data excluding fields that require admin privileges.

4.8.2 Stimulus/Response Sequences

- User navigates to a member profile and clicks Edit -> System displays the editable form.
- User submits valid changes -> System saves the changes and displays a confirmation.
- Member attempts to edit a restricted field (e.g., subscription status) -> System ignores the field or returns 403.

4.8.3 Functional Requirements

REQ-27: Members may edit: first name, last name, phone number, and password.

REQ-28: Administrators may edit all profile fields including subscription status and role.

REQ-29: All changes must be validated before persistence; invalid data must return clear field-level error messages.

4.9 View Member Profile

4.9.1 Description and Priority

Priority: High (H)

Administrators must be able to view full details of any member profile. Members must be able to view their own profile.

4.9.2 Stimulus/Response Sequences

- User navigates to a member profile -> System displays all relevant profile details.
- Member navigates to another member's profile URL -> System returns 403 Forbidden.

4.9.3 Functional Requirements

REQ-30: The member profile view must display: personal details, subscription status, active subscription summary, and registration date.

REQ-31: Administrators must additionally see: payment history link, reservation history link, and account status (active/restricted/locked).

4.10 Deactivate Member Account

4.10.1 Description and Priority

Priority: High (H)

Administrators must be able to deactivate a member account. Deactivated accounts cannot log in and cannot make new reservations. Deactivation is reversible.

4.10.2 Stimulus/Response Sequences

- Administrator selects a member and clicks Deactivate -> System displays a confirmation modal.
- Administrator confirms -> System sets the account to inactive, cancels future reservations, sends notification email.
- Deactivated member attempts to log in -> System displays a message indicating the account is inactive.

4.10.3 Functional Requirements

REQ-32: Deactivating an account must set the user's `is_active` flag to false.

REQ-33: Future reservations of a deactivated member must be automatically cancelled with a notification sent to the member.

REQ-34: Administrators must be able to reactivate a previously deactivated account.

4.11 Search Members

4.11.1 Description and Priority

Priority: Medium (M)

Administrators must be able to search the member database using text-based queries across multiple fields to quickly locate a specific member.

4.11.2 Stimulus/Response Sequences

- Administrator types in the search bar -> System performs a live search as the user types (debounced).
- Search returns results -> System displays a paginated list of matching members.
- No results found -> System displays a clear 'no members found' message.

4.11.3 Functional Requirements

REQ-35: The search must query across: first name, last name, email address, and phone number.

REQ-36: Search must be case-insensitive and support partial matches.

REQ-37: Results must be displayed in a paginated list with a default of 20 items per page.

4.12 Filter Members by Status

4.12.1 Description and Priority

Priority: Medium (M)

Administrators must be able to filter the member list by subscription and account status to efficiently manage different cohorts of members.

4.12.2 Stimulus/Response Sequences

- Administrator opens the member list -> System displays filter controls.
- Administrator selects a filter (e.g., 'Restricted') -> System refreshes the list showing only matching members.
- Administrator clears filters -> System displays the unfiltered list.

4.12.3 Functional Requirements

REQ-38: The filter must support the following statuses: Active, Expired, Restricted, and Inactive (deactivated).

REQ-39: Multiple filters may be active simultaneously.

REQ-40: The active filter state must be reflected in the URL query parameters so the view can be bookmarked.

4.13 Export Member List to CSV

4.13.1 Description and Priority

Priority: Low (L)

Administrators must be able to export the current (filtered or unfiltered) member list to a CSV file for use in external tools such as spreadsheets.

4.13.2 Stimulus/Response Sequences

- Administrator clicks 'Export to CSV' on the member list page -> System generates and downloads the file.
- Export is triggered on a filtered list -> Only filtered members are included in the CSV.

4.13.3 Functional Requirements

REQ-41: The exported CSV must include: first name, last name, email, phone, subscription status, and registration date.

REQ-42: The export must reflect any active search or filter criteria.

REQ-43: The filename must include the export date (e.g., members_2025-03-15.csv).

4.14 Restricted Members List

4.14.1 Description and Priority

Priority: Medium (M)

Administrators must have a dedicated view listing all members whose subscription has expired and who have been automatically moved to restricted status.

4.14.2 Stimulus/Response Sequences

- Administrator navigates to Members > Restricted -> System displays all restricted members.
- Administrator selects a restricted member -> System shows options to renew their subscription or deactivate their account.

4.14.3 Functional Requirements

REQ-44: A member is restricted when their subscription's end_date is in the past and status has not been manually renewed.

REQ-45: Restricted members must appear in a dedicated list, separate from the main active member list.

REQ-46: The list must show: member name, email, subscription end date, and days since expiry.

Subscription Management (Features 4.15 - 4.21)

4.15 Create Subscription

4.15.1 Description and Priority

Priority: High (H)

Administrators must be able to create a new subscription for a member, selecting a subscription type and setting the start date.

4.15.2 Stimulus/Response Sequences

- Administrator navigates to a member profile and clicks Add Subscription -> System shows the subscription form.
- Administrator selects a type and start date -> System calculates end date and total amount, displays a preview.
- Administrator confirms -> System creates the subscription record and updates the member's status to active.

4.15.3 Functional Requirements

REQ-47: A subscription must be linked to exactly one member and one subscription type.

REQ-48: The system must automatically calculate the end date based on the type's duration.

REQ-49: A member may not have two overlapping active subscriptions.

REQ-50: Creating a subscription must update the member's subscription_status to 'active'.

4.16 View Subscription Details

4.16.1 Description and Priority

Priority: High (H)

Both members and administrators must be able to view the full details of a subscription including pricing breakdown, dates, and current status.

4.16.2 Stimulus/Response Sequences

- User navigates to the subscription details page -> System displays full subscription information.
- Member navigates to another member's subscription URL -> System returns 403 Forbidden.

4.16.3 Functional Requirements

REQ-51: The detail view must show: subscription type, start date, end date, base fee, number of PT sessions (if personalized), total amount, and current status.

REQ-52: If the subscription is personalized, the pricing formula breakdown must be displayed: Base Fee + (PT Sessions x 50 RON).

4.17 Renew Subscription

4.17.1 Description and Priority

Priority: High (H)

Administrators must be able to renew a member's expired or soon-to-expire subscription, creating a new subscription record that begins at the end of the current one.

4.17.2 Stimulus/Response Sequences

- Administrator clicks Renew on a member's subscription -> System pre-populates the renewal form with the same subscription type.
- Administrator confirms renewal -> System creates a new subscription starting the day after the current end date.

4.17.3 Functional Requirements

REQ-53: Renewal must create a new subscription record; it must not modify the existing record.

REQ-54: The new subscription's start date must default to the day after the current subscription's end date.

REQ-55: The member's status must be updated to 'active' upon renewal even if it was 'restricted'.

4.18 Cancel Subscription

4.18.1 Description and Priority

Priority: Medium (M)

Administrators must be able to cancel an active subscription, setting it to a cancelled state and moving the member to restricted status.

4.18.2 Stimulus/Response Sequences

- Administrator selects an active subscription and clicks Cancel -> System shows a confirmation modal with impact summary.
- Administrator confirms -> System sets subscription status to 'cancelled' and member status to 'restricted'.

4.18.3 Functional Requirements

REQ-56: Cancellation must set the subscription status to 'cancelled' and record the cancellation date.

REQ-57: Cancellation must update the member's subscription_status to 'restricted'.

REQ-58: Future reservations made under the cancelled subscription must be automatically cancelled.

4.19 Personalized Subscription Pricing

4.19.1 Description and Priority

Priority: High (H)

Administrators must be able to configure a personalized subscription with a variable number of personal training sessions. The total price must be calculated by the system.

4.19.2 Stimulus/Response Sequences

- Administrator selects the 'Personalized' subscription type -> System reveals a PT sessions field.
- Administrator enters the number of PT sessions -> System live-calculates and displays the total.
- Administrator confirms -> System stores the base fee, PT session count, and computed total.

4.19.3 Functional Requirements

REQ-59: The pricing formula is: $\text{Total} = \text{Base Fee} + (\text{Number of PT Sessions} \times 50 \text{ RON})$.

REQ-60: The PT session count must be a positive integer.

REQ-61: The system must store both the components (base fee, PT sessions) and the computed total separately to support receipt generation.

4.20 Automatic Subscription Expiry

4.20.1 Description and Priority

Priority: High (H)

The system must automatically detect subscriptions that have passed their end date and transition the associated member to restricted status without requiring administrator action.

4.20.2 Stimulus/Response Sequences

- System scheduled job runs daily at midnight -> System queries all subscriptions where $\text{end_date} < \text{today}$ and $\text{status} = \text{'active'}$.
- Expired subscriptions found -> System updates each to 'expired', updates member status to 'restricted', sends expiry notification email.

4.20.3 Functional Requirements

REQ-62: A background scheduled job must run daily to check for expired subscriptions.

REQ-63: Any active subscription whose end_date is before the current date must be set to 'expired'.

REQ-64: The member associated with an expired subscription must be automatically set to 'restricted' status.

REQ-65: An expiry notification email must be sent to the member on the day their subscription expires.

4.21 Subscription Type Configuration

4.21.1 Description and Priority

Priority: Medium (M)

Administrators must be able to define and manage the catalog of subscription types available to assign to members.

4.21.2 Stimulus/Response Sequences

- Administrator navigates to Settings > Subscription Types -> System lists all defined types.
- Administrator clicks Add Type -> System displays a form for name, duration (days), and base fee.
- Administrator saves -> System adds the type to the catalog, making it available for new subscriptions.

4.21.3 Functional Requirements

REQ-66: Each subscription type must have: a name, a duration in days, and a base fee in RON.

REQ-67: A subscription type that is in use by at least one active subscription must not be deletable; it may only be archived.

REQ-68: Archived types must not appear in the new subscription form but must remain readable for historical records.

Payment Processing (Features 4.22 - 4.26)

4.22 Online Payment via Gateway

4.22.1 Description and Priority

Priority: High (H)

Members must be able to pay for their subscriptions online using a credit or debit card through an integrated payment gateway. The system must never handle or store raw card data.

4.22.2 Stimulus/Response Sequences

- Member navigates to their subscription and clicks Pay Online -> System initiates a checkout session via the gateway API.
- Member completes payment on gateway UI -> Gateway sends a webhook to the system confirming payment.
- System receives successful webhook -> System records the payment, activates the subscription.
- Payment fails -> Gateway notifies system; system updates payment status and notifies the member.

4.22.3 Functional Requirements

REQ-69: Card data must be collected exclusively through the payment gateway's certified JavaScript SDK or hosted payment page.

REQ-70: The system must store the gateway's payment reference ID but must never store card numbers, CVVs, or expiry dates.

REQ-71: Payment status must be updated via webhook, not based on client-side redirect alone.

REQ-72: The system must handle the following payment statuses: pending, succeeded, failed, and refunded.

4.23 View Payment History

4.23.1 Description and Priority

Priority: High (H)

Members must be able to view a chronological list of all payments made. Administrators must be able to view payment history for any member.

4.23.2 Stimulus/Response Sequences

- User navigates to Payment History -> System displays a paginated list of transactions.
- User clicks a transaction -> System displays the full transaction details and a link to the receipt.

4.23.3 Functional Requirements

REQ-73: Each entry in the payment history must show: payment date, subscription type, amount, currency, and status.

REQ-74: The list must be sorted by date descending by default.

REQ-75: Members must only see their own payment history; administrators can view any member's history.

4.24 Payment Receipt Generation

4.24.1 Description and Priority

Priority: Medium (M)

The system must automatically generate and deliver a payment receipt to the member after each successful payment. Members must also be able to download receipts on demand.

4.24.2 Stimulus/Response Sequences

- Payment is confirmed as succeeded -> System generates a PDF receipt and emails it to the member.
- Member navigates to a past payment -> System provides a Download Receipt button that generates the PDF on demand.

4.24.3 Functional Requirements

REQ-76: Each receipt must include: receipt number, date, member name, subscription type, pricing breakdown, total amount, and payment gateway reference.

REQ-77: Receipts must be downloadable as PDF files.

REQ-78: A copy of the receipt must be sent to the member's email address within 2 minutes of payment confirmation.

4.25 Monthly Revenue Report

4.25.1 Description and Priority

Priority: Medium (M)

Administrators must be able to generate a report summarizing all payments received within a selected month, grouped by subscription type.

4.25.2 Stimulus/Response Sequences

- Administrator navigates to Reports > Revenue -> System displays month/year selectors.
- Administrator selects a month and clicks Generate -> System aggregates data and displays the report.
- Administrator clicks Export -> System downloads the report as a CSV file.

4.25.3 Functional Requirements

REQ-79: The report must aggregate: total revenue for the selected month, revenue broken down by subscription type, and number of payments.

REQ-80: The report must be filterable by month and year.

REQ-81: The report must be exportable as a CSV file.

4.26 Refund Request Tracking

4.26.1 Description and Priority

Priority: Low (L)

Administrators must be able to record and track manual refund requests. The actual refund is processed outside the system (via the payment gateway dashboard) but must be logged.

4.26.2 Stimulus/Response Sequences

- Administrator navigates to a payment record and clicks Log Refund -> System displays a refund form.
- Administrator enters refund amount and reason and saves -> System updates payment status to 'refunded' and stores the note.

4.26.3 Functional Requirements

REQ-82: A refund record must store: original payment ID, refund amount, reason, refund date, and the admin who logged it.

REQ-83: A payment may only be marked as refunded once.

REQ-84: Refunded payments must be excluded from revenue report totals.

Training Class Management (Features 4.27 - 4.32)

4.27 Create Training Class

4.27.1 Description and Priority

Priority: High (H)

Administrators must be able to create new group training class sessions, defining the trainer, facility, time, and capacity.

4.27.2 Stimulus/Response Sequences

- Administrator navigates to Classes > Add New -> System displays the class creation form.
- Administrator fills in all required fields and saves -> System creates the class record and makes it visible to members for reservation.
- Required field is missing -> System highlights the field and prevents submission.

4.27.3 Functional Requirements

REQ-85: A class record must contain: name, description, trainer (FK), facility (FK), start time (datetime), end time (datetime), and capacity (max participants).

REQ-86: The system must prevent creating a class in a facility if another class is already scheduled in the same facility at an overlapping time.

REQ-87: The system must prevent assigning a trainer to a class if they already have another class at an overlapping time.

4.28 Edit Training Class Details

4.28.1 Description and Priority

Priority: High (H)

Administrators must be able to modify the details of an existing training class, with appropriate handling of existing reservations when capacity is reduced.

4.28.2 Stimulus/Response Sequences

- Administrator edits a class and reduces capacity below the current reservation count -> System warns and prevents the change.
- Administrator edits a class and changes the time -> System sends update notifications to all members with active reservations.
- Administrator saves valid changes -> System updates the record and confirms.

4.28.3 Functional Requirements

REQ-88: The new capacity must not be less than the current number of active reservations for that class.

REQ-89: If the class start or end time is changed, all members with active reservations must receive an email notification of the change.

REQ-90: Editing a class must not affect the status of existing reservations.

4.29 Cancel Training Class

4.29.1 Description and Priority

Priority: High (H)

Administrators must be able to cancel a training class, automatically cancelling all associated reservations and notifying affected members.

4.29.2 Stimulus/Response Sequences

- Administrator selects a class and clicks Cancel -> System shows a confirmation modal listing the number of affected reservations.
- Administrator confirms -> System cancels the class and all reservations, sends cancellation emails to all affected members.

4.29.3 Functional Requirements

REQ-91: Cancelling a class must set its status to 'cancelled'.

REQ-92: All active reservations for the cancelled class must be set to 'cancelled'.

REQ-93: Each member with a cancelled reservation must receive an email notification within 5 minutes of the class cancellation.

4.30 View Class Schedule

4.30.1 Description and Priority

Priority: Medium (M)

All authenticated users must be able to view the upcoming class schedule, with the display adapted to their role (e.g., members see available spots; trainers see their own classes).

4.30.2 Stimulus/Response Sequences

- User navigates to the Schedule page -> System displays upcoming classes in a calendar or list view.
- User switches between weekly and list view -> System re-renders the schedule in the selected format.
- Member clicks a class -> System shows class details and the option to reserve if spots are available.

4.30.3 Functional Requirements

REQ-94: The schedule must display classes for at least the next 4 weeks.

REQ-95: Each class entry must show: name, trainer, facility, start time, end time, and available spots out of total capacity.

REQ-96: Past classes must not be shown in the default view but must be accessible via a date range filter.

REQ-97: Trainer view must highlight only their assigned classes.

4.31 Class Capacity Enforcement

4.31.1 Description and Priority

Priority: High (H)

The system must enforce the maximum participant limit for each class, preventing reservations beyond capacity and keeping available spot counts accurate.

4.31.2 Stimulus/Response Sequences

- Member attempts to reserve a full class -> System rejects the request with a 'class is full' message and offers the waitlist.
- A reservation is cancelled -> System increments the available spots count and promotes the first waitlisted member if applicable.

4.31.3 Functional Requirements

REQ-98: The system must atomically check and decrement available spots when a reservation is created to prevent race conditions.

REQ-99: A class is considered full when current active reservations equal the class capacity field.

REQ-100: Available spot counts must be updated in real time; the schedule view must not show stale counts.

4.32 Assign Trainer to Class

4.32.1 Description and Priority

Priority: Medium (M)

Administrators must be able to assign a trainer to a class and reassign to a different trainer if needed, with conflict detection.

4.32.2 Stimulus/Response Sequences

- Administrator selects a trainer from the dropdown when creating or editing a class -> System validates availability.
- Selected trainer has a scheduling conflict -> System displays a conflict warning and prevents saving.
- Assignment is valid -> System saves and the class appears on the trainer's schedule view.

4.32.3 Functional Requirements

REQ-101: The system must check for scheduling conflicts before confirming a trainer assignment.

REQ-102: A trainer may not be assigned to two classes with overlapping time slots.

REQ-103: Reassigning a trainer to a class must send a notification to both the old and new trainer.

Reservation System (Features 4.33 - 4.39)

4.33 Make a Class Reservation

4.33.1 Description and Priority

Priority: High (H)

Active members must be able to reserve a spot in any upcoming class that has available capacity. Restricted or inactive members must not be able to make reservations.

4.33.2 Stimulus/Response Sequences

- Active member clicks Reserve on a class with available spots -> System creates the reservation and shows confirmation.
- Member with restricted status attempts to reserve -> System rejects with a message explaining the status.
- Member already has a reservation for the same class -> System rejects with a duplicate reservation error.

4.33.3 Functional Requirements

REQ-104: Only members with an 'active' subscription status may make reservations.

REQ-105: A member may not have more than one active reservation for the same class.

REQ-106: The system must atomically reserve the spot to prevent double-booking under concurrent requests.

REQ-107: A confirmation notification must be sent immediately after a successful reservation.

4.34 Cancel a Reservation

4.34.1 Description and Priority

Priority: High (H)

Members must be able to cancel their own reservations. Administrators must be able to cancel any reservation.

4.34.2 Stimulus/Response Sequences

- Member clicks Cancel on an active reservation -> System shows a confirmation and cancels upon confirmation.
- Cancellation is made less than 2 hours before class start -> System cancels but logs a late cancellation flag.
- Reservation is cancelled -> System increments available spots and notifies the next waitlisted member.

4.34.3 Functional Requirements

REQ-108: Cancellation must set the reservation status to 'cancelled' and record the cancellation timestamp.

REQ-109: Cancellations made within 2 hours of the class start time must be flagged as late cancellations.

REQ-110: On cancellation, the system must check the waitlist for that class and offer the spot to the next member.

4.35 View Own Reservations

4.35.1 Description and Priority

Priority: High (H)

Members must be able to view all their upcoming and past reservations in a personal reservation history.

4.35.2 Stimulus/Response Sequences

- Member navigates to My Reservations -> System displays upcoming reservations followed by past reservations.
- Member clicks a reservation -> System shows full class details for that reservation.

4.35.3 Functional Requirements

REQ-111: The reservation list must be split into two sections: Upcoming and Past.

REQ-112: Each entry must show: class name, trainer, facility, date and time, and reservation status.

REQ-113: Upcoming reservations must include a Cancel button if cancellation is still permitted.

4.36 View All Reservations for a Class

4.36.1 Description and Priority

Priority: Medium (M)

Administrators and Trainers must be able to view the full participant list for a specific class.

4.36.2 Stimulus/Response Sequences

- User navigates to a class detail page -> System displays the list of all members who have active reservations.
- Trainer views the list -> System also shows attendance tracking controls.

4.36.3 Functional Requirements

REQ-114: The participant list must show: member name, email, reservation date, and reservation status.

REQ-115: The list must be downloadable as a CSV file by administrators.

REQ-116: The total participant count and remaining capacity must be displayed at the top of the list.

4.37 Waitlist Management

4.37.1 Description and Priority

Priority: Medium (M)

When a class is full, members must be able to join a waitlist. When a spot becomes available, the first waitlisted member must be automatically notified.

4.37.2 Stimulus/Response Sequences

- Member attempts to reserve a full class -> System offers to add them to the waitlist.
- Member accepts -> System adds them to the waitlist and confirms their position.

- A reservation is cancelled -> System promotes the first waitlisted member, notifies them, and gives them 2 hours to confirm.

4.37.3 Functional Requirements

REQ-117: The waitlist must be ordered strictly by the time the member joined.

REQ-118: When promoted from the waitlist, the member must receive an email and/or SMS notification with a 2-hour confirmation window.

REQ-119: If the promoted member does not confirm within 2 hours, the system must offer the spot to the next waitlisted member.

REQ-120: Members must be able to remove themselves from a waitlist.

4.38 Reservation Confirmation Notification

4.38.1 Description and Priority

Priority: High (H)

Upon a successful reservation, the system must immediately send a confirmation notification to the member containing all relevant class details.

4.38.2 Stimulus/Response Sequences

- Reservation is created -> System triggers an email (and optional SMS) notification immediately.
- Notification delivery fails -> System logs the failure and retries up to 3 times.

4.38.3 Functional Requirements

REQ-121: The confirmation notification must be sent within 1 minute of the reservation being created.

REQ-122: The email must include: class name, trainer name, facility name, date, start time, end time, and a cancellation link.

REQ-123: The notification must be sent via email; SMS is optional and only sent if the member has a phone on file and has opted in.

4.39 Reservation Reminder Notification

4.39.1 Description and Priority

Priority: Medium (M)

The system must send automated reminder notifications to members before their reserved class starts to reduce no-shows.

4.39.2 Stimulus/Response Sequences

- System scheduler runs hourly -> System identifies reservations starting in approximately 24 hours.
- Matching reservations found -> System sends reminder emails to all affected members.

4.39.3 Functional Requirements

REQ-124: A reminder notification must be sent 24 hours before the class start time.

REQ-125: The reminder must include: class name, trainer, facility, start time, and a cancellation link.

REQ-126: Members who have already cancelled their reservation must not receive a reminder.

REQ-127: Members must be able to opt out of reminder notifications from their profile settings.

Trainer Management (Features 4.40 - 4.44)

4.40 Create Trainer Profile

4.40.1 Description and Priority

Priority: High (H)

Administrators must be able to register new trainers, creating both a user account with Trainer role and the associated trainer profile.

4.40.2 Stimulus/Response Sequences

- Administrator navigates to Trainers > Add New -> System displays the trainer creation form.
- Administrator submits valid data -> System creates user account and trainer profile, sends welcome email with password-set link.
- Duplicate email submitted -> System displays a validation error.

4.40.3 Functional Requirements

REQ-128: The trainer creation form must capture: first name, last name, email address, phone number, and specializations.

REQ-129: A welcome email with a password-set link must be sent to the trainer upon account creation.

REQ-130: The new trainer account must have the 'trainer' role assigned by default.

4.41 Edit Trainer Profile

4.41.1 Description and Priority

Priority: Medium (M)

Administrators must be able to update any trainer's profile. Trainers must be able to update their own bio and contact details.

4.41.2 Stimulus/Response Sequences

- User navigates to a trainer profile and clicks Edit -> System displays the editable form for allowed fields.
- Trainer attempts to edit another trainer's profile -> System returns 403 Forbidden.
- Valid changes are submitted -> System saves and confirms.

4.41.3 Functional Requirements

REQ-131: Trainers may edit: phone number, bio/description, and profile photo.

REQ-132: Administrators may edit all fields including specializations and active status.

4.42 Deactivate Trainer Account

4.42.1 Description and Priority

Priority: Medium (M)

Administrators must be able to deactivate a trainer's account. Deactivated trainers cannot log in and must not be assignable to new classes.

4.42.2 Stimulus/Response Sequences

- Administrator selects a trainer and clicks Deactivate -> System shows a confirmation modal listing upcoming classes affected.
- Administrator confirms -> System deactivates the account and flags upcoming classes assigned to this trainer as needing reassignment.

4.42.3 Functional Requirements

REQ-133: Deactivation must set the trainer's `is_active` flag to false.

REQ-134: Upcoming classes assigned to a deactivated trainer must be flagged with a 'trainer missing' status and appear in an admin alert dashboard.

REQ-135: A deactivated trainer must not appear in the trainer selection dropdown when creating or editing a class.

4.43 View Trainer Schedule

4.43.1 Description and Priority

Priority: Medium (M)

Trainers must be able to view their own upcoming class schedule. Administrators must be able to view any trainer's schedule.

4.43.2 Stimulus/Response Sequences

- Trainer navigates to My Schedule -> System displays their upcoming classes in chronological order.
- Administrator navigates to a trainer's profile and clicks View Schedule -> System displays the same view.

4.43.3 Functional Requirements

REQ-136: The schedule view must show: class name, facility, start time, end time, and current participant count.

REQ-137: The schedule must be viewable as a list (default) or a weekly calendar.

REQ-138: The schedule must cover at least the next 30 days.

4.44 Trainer Availability Management

4.44.1 Description and Priority

Priority: Low (L)

Trainers must be able to define recurring availability windows. Administrators use these windows as a guide when scheduling classes.

4.44.2 Stimulus/Response Sequences

- Trainer navigates to My Availability -> System displays a weekly availability grid.
- Trainer marks available blocks -> System saves the preferences.
- Administrator creates a class -> System shows a soft warning if the assigned trainer has not marked availability for that slot.

4.44.3 Functional Requirements

REQ-139: Availability is stored as a weekly recurring schedule (day of week + time range).

REQ-140: Availability is advisory only; it does not prevent class assignment outside available hours.

REQ-141: The availability grid must cover the full 7-day week in 30-minute intervals.

Facility Management (Features 4.45 - 4.48)

4.45 Create Facility

4.45.1 Description and Priority

Priority: High (H)

Administrators must be able to define the physical rooms or areas within the fitness center that can be assigned to training classes.

4.45.2 Stimulus/Response Sequences

- Administrator navigates to Facilities > Add New -> System displays the facility creation form.
- Administrator submits valid data -> System creates the facility and makes it available for class assignment.

4.45.3 Functional Requirements

REQ-142: A facility record must contain: name, description, and maximum capacity (persons).

REQ-143: Facility names must be unique within the system.

REQ-144: Newly created facilities must be immediately available in the class creation trainer dropdown.

4.46 Edit Facility Details

4.46.1 Description and Priority

Priority: Medium (M)

Administrators must be able to update facility details. Reducing capacity below the maximum participant count of any scheduled class must be prevented.

4.46.2 Stimulus/Response Sequences

- Administrator edits a facility and reduces capacity -> System checks scheduled classes and warns if any exceed the new limit.
- Administrator saves valid changes -> System updates the record and confirms.

4.46.3 Functional Requirements

REQ-145: The system must prevent saving a new capacity value if any future scheduled class for that facility has more reservations than the proposed new capacity.

REQ-146: Changes to the facility name must be reflected everywhere the facility is referenced in the UI.

4.47 Facility Occupancy Report

4.47.1 Description and Priority

Priority: Medium (M)

Administrators must be able to generate a report showing the utilization rate of each facility over a selected date range.

4.47.2 Stimulus/Response Sequences

- Administrator navigates to Reports > Occupancy and selects a date range -> System generates the report.
- Administrator clicks Export -> System downloads the report as a CSV.

4.47.3 Functional Requirements

REQ-147: The report must show per facility: total scheduled class hours, total available participant slots, total reservations made, and occupancy rate (reservations / slots as a percentage).

REQ-148: The report must be filterable by date range and by individual facility.

REQ-149: The report must be exportable as CSV.

4.48 Facility Availability Calendar

4.48.1 Description and Priority

Priority: Low (L)

Administrators must be able to view a calendar showing which facilities are booked at any given time to avoid scheduling conflicts.

4.48.2 Stimulus/Response Sequences

- Administrator navigates to Facilities > Calendar -> System displays a weekly calendar with facility bookings as color-coded blocks.

- Administrator clicks a booking block -> System shows class details for that booking.
- Administrator navigates to the next week -> System loads the following week's schedule.

4.48.3 Functional Requirements

REQ-150: The calendar must display all facilities side by side for the selected week.

REQ-151: Each booked time block must display: class name and trainer name.

REQ-152: Free (unbooked) slots must be visually distinct from booked slots.

Reporting and Notifications (Features 4.49 - 4.50)

4.49 Member Activity Report

4.49.1 Description and Priority

Priority: Low (L)

Administrators must be able to generate a report showing the reservation and attendance activity of all members over a selected period, to identify engaged and inactive members.

4.49.2 Stimulus/Response Sequences

- Administrator navigates to Reports > Member Activity and selects a date range -> System generates the report.
- Administrator clicks Export -> System downloads the report as a CSV.

4.49.3 Functional Requirements

REQ-153: The report must show per member: total reservations made, total cancellations, and late cancellation count within the selected period.

REQ-154: The report must be sortable by any column.

REQ-155: The report must be filterable by date range and by subscription status.

REQ-156: The report must be exportable as CSV.

4.50 Subscription Expiry Notifications

4.50.1 Description and Priority

Priority: Medium (M)

The system must proactively notify members when their subscription is approaching its expiry date, giving them time to renew before being restricted.

4.50.2 Stimulus/Response Sequences

- System scheduler runs daily -> System queries subscriptions expiring in exactly 7 days and exactly 1 day.
- Matching subscriptions found -> System sends a personalized renewal reminder email to each affected member.

4.50.3 Functional Requirements

REQ-157: Expiry reminder emails must be sent at 7 days and again at 1 day before the subscription end date.

REQ-158: The reminder email must include: the expiry date, the subscription type name, and a direct link to the renewal or payment page.

REQ-159: If a member renews their subscription before the reminder is due, the reminder must not be sent.

REQ-160: Members must be able to opt out of expiry reminder notifications from their profile settings.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system must meet the following performance targets under normal operating load, defined as up to 100 concurrent users:

- API response time for read operations (GET) must be under 500ms at the 95th percentile.
- API response time for write operations (POST, PUT, DELETE) must be under 1000ms at the 95th percentile.
- Report generation (revenue and occupancy) must complete within 10 seconds.
- The login endpoint must handle at least 50 concurrent requests without returning errors.
- CSV exports for member lists of up to 10,000 records must complete within 30 seconds.
- Notification emails must be queued and delivered within 2 minutes of the triggering event.

5.2 Safety Requirements

The system must protect against data loss and ensure continuity of service:

- The database must be backed up automatically every 24 hours. Backups must be retained for a minimum of 30 days.
- In the event of a critical application failure, the system must be recoverable to its last consistent state within a Recovery Time Objective (RTO) of 4 hours.
- The Recovery Point Objective (RPO) must not exceed 24 hours, meaning at most 24 hours of data may be lost in a worst-case failure scenario.
- Database transactions must be atomic and use appropriate isolation levels to prevent partial updates and data corruption.

5.3 Security Requirements

The system must implement the following security controls:

- All passwords must be stored as bcrypt hashes with a minimum cost factor of 12. Plaintext passwords must never be logged or stored.
- JWT tokens must be signed using RS256 (asymmetric) or HS256 with a secret of at least 256 bits.
- All API endpoints must validate and sanitize inputs to prevent SQL injection, XSS, and similar injection attacks. SQLAlchemy ORM parameterization must be used exclusively.
- The system must implement rate limiting on the authentication endpoints: a maximum of 10 requests per IP per minute.
- All communications must use HTTPS with TLS 1.2 or higher. HTTP must redirect to HTTPS.
- The system must comply with GDPR. A data deletion endpoint must allow administrators to fully remove a member's personal data upon request.
- Sensitive configuration values (database credentials, JWT secrets, payment gateway keys) must be loaded from environment variables and must not appear in source code or version control.

5.4 Software Quality Attributes

- **Availability:** The system must target 99.5% uptime per calendar month, excluding planned maintenance windows. Planned maintenance must be scheduled outside peak hours (06:00-22:00 local time).
- **Maintainability:** Backend code must follow PEP 8 and be linted with ruff or flake8. Frontend code must follow the project's ESLint configuration. All modules must have a test coverage of at least 70%.
- **Portability:** The entire application stack must be runnable locally using Docker Compose with a single command (docker compose up). This includes the database, backend, and frontend.
- **Usability:** A new member with no prior training must be able to complete a class reservation within 3 minutes of first login. The interface must comply with WCAG 2.1 Level AA.
- **Scalability:** The backend must be stateless to allow horizontal scaling. No session state may be stored in the application process memory.
- **Testability:** All business logic must reside in service-layer functions independent of the web framework to facilitate unit testing without spinning up the full HTTP server.

5.5 Business Rules

- A member with a status of 'restricted' or 'inactive' may not make new class reservations.
- A member may not hold two overlapping active subscriptions simultaneously.
- Subscription prices may only be modified by an Administrator.
- A personalized subscription fee is calculated as: Base Fee + (Number of PT Sessions x 50 RON). This formula is fixed for v1.0.
- A class reservation may only be made for a class scheduled in the future. Past classes cannot be retroactively reserved.
- Late cancellations (within 2 hours of class start) are allowed but flagged. The business may use this data for policy enforcement at their discretion.
- A trainer may not be double-booked across two simultaneous classes.
- A facility may not host two simultaneous classes.

6. Other Requirements

The system must store and display all dates and times in the Europe/Bucharest timezone (UTC+2/UTC+3 DST). The currency for all monetary values is RON (Romanian Leu). All monetary values must be stored as integers in bani (1/100 RON) to avoid floating-point precision issues. The primary interface language for v1.0 is English; Romanian localization is a deferred enhancement. All log output must be structured JSON and must include at minimum: timestamp, log level, service name, request ID, and message. Logs must not contain personally identifiable information (PII) such as email addresses or phone numbers in plaintext.

7. Technical Architecture

7.1 Overview

The system follows a decoupled client-server architecture. The React frontend communicates exclusively with the FastAPI backend via a versioned REST API. The backend persists all data in PostgreSQL via SQLAlchemy. Third-party integrations (payment gateway, email, SMS) are called exclusively from the backend.

Layer	Technology	Responsibility
Frontend	React (JavaScript)	SPA, routing, state, API calls
Backend	Python 3.11 / FastAPI	REST API, business logic, auth, validation
Database	PostgreSQL 15+	Persistent relational storage
ORM / Migrations	SQLAlchemy 2.0 / Alembic	DB access layer and schema versioning
Authentication	JWT (python-jose + passlib)	Stateless auth, role-based access
Payments	Stripe or PayU SDK	Online payment processing
Email	SendGrid / AWS SES	Transactional notifications
SMS (optional)	Twilio	Reservation and expiry reminders
Task Scheduler	APScheduler or Celery + Redis	Daily expiry checks, reminder dispatch

7.2 Backend

The backend exposes all endpoints under the **/api/v1** prefix. Pydantic v2 models handle request validation and response serialization. SQLAlchemy async sessions are used for all database operations. Authentication middleware validates the JWT on every protected request and injects the current user into the request context.

Key packages:

- fastapi, uvicorn[standard] -- web framework and ASGI server
- sqlalchemy[asyncio], asyncpg, alembic -- async ORM, driver, and migrations
- pydantic-settings -- environment-based configuration management
- python-jose[cryptography], passlib[bcrypt] -- JWT and password hashing
- apscheduler -- in-process task scheduling for daily jobs
- pytest, httpx, pytest-asyncio -- async-compatible testing

7.3 Frontend

The React SPA is served as static assets. JWT access tokens are stored in memory only (never localStorage or cookies) and refreshed silently via the refresh token. All API calls go through a centralized Axios instance that attaches the Authorization header and handles 401 responses by triggering a token refresh.

Key packages:

- react, react-router-dom -- UI and client-side routing
- axios -- HTTP client with interceptors for auth and error handling
- @tanstack/react-query -- server state management, caching, and background refetching
- react-hook-form, zod -- form state management and schema-based validation
- tailwindcss -- utility-first styling
- date-fns -- date formatting and manipulation

7.4 Database Schema (Core Entities)

All tables include: id (UUID, primary key), created_at (timestampz), updated_at (timestampz).

Table	Key Columns
users	email (unique), hashed_password, role ENUM(admin member trainer), is_active
members	user_id FK, first_name, last_name, phone, date_of_birth, subscription_status ENUM
trainers	user_id FK, first_name, last_name, phone, bio, specializations TEXT[]
subscription_types	name (unique), duration_days, base_fee_bani, is_archived
subscriptions	member_id FK, type_id FK, base_fee_bani, pt_sessions, total_amount_bani, start_date, end_date, status ENUM
payments	subscription_id FK, amount_bani, currency, gateway_ref (unique), status ENUM, paid_at, refunded_at, refund_note
facilities	name (unique), description, capacity
classes	trainer_id FK, facility_id FK, name, description, start_time, end_time, capacity, status ENUM
reservations	member_id FK, class_id FK, status ENUM, is_late_cancel, waitlist_position
trainer_availability	trainer_id FK, day_of_week INT, start_time TIME, end_time TIME

7.5 API Structure

All endpoints are prefixed with **/api/v1**. The main resource groups are:

Resource	Endpoints	Access
Auth	POST /auth/login POST /auth/refresh POST /auth/forgot-password POST /auth/reset-password	Public
Members	GET /members, POST /members GET /members/{id}, PUT /members/{id} DELETE /members/{id} GET /members/{id}/reservations GET /members/restricted	Admin; Member (own only)
Subscriptions	GET /subscriptions, POST /subscriptions GET /subscriptions/{id} POST /subscriptions/{id}/renew DELETE /subscriptions/{id}	Admin; Member (own)

Resource	Endpoints	Access
Payments	POST /payments/checkout GET /payments/history GET /payments/{id}/receipt POST /payments/{id}/refund	Admin; Member (own)
Classes	GET /classes, POST /classes GET /classes/{id}, PUT /classes/{id} DELETE /classes/{id} GET /classes/{id}/reservations	Admin; Trainer (own); Member (read)
Reservations	GET /reservations, POST /reservations DELETE /reservations/{id} POST /reservations/{id}/waitlist	Admin; Member (own)
Trainers	GET /trainers, POST /trainers GET /trainers/{id}, PUT /trainers/{id} GET /trainers/{id}/schedule PUT /trainers/{id}/availability	Admin; Trainer (own)
Facilities	GET /facilities, POST /facilities GET /facilities/{id}, PUT /facilities/{id} GET /facilities/calendar	Admin
Reports	GET /reports/revenue GET /reports/occupancy GET /reports/member-activity	Admin
Subscription Types	GET /subscription-types, POST /subscription-types PUT /subscription-types/{id}	Admin

7.6 Project Structure

Recommended monorepo layout:

```

fitness-app/
  backend/
    app/
      api/
        v1/
          routes/      auth.py  members.py  subscriptions.py
                       payments.py  classes.py  reservations.py
                       trainers.py  facilities.py  reports.py
          dependencies.py  get_current_user, require_role
          router.py       aggregates all route prefixes
        core/
          config.py       pydantic-settings BaseSettings
          security.py     JWT encode/decode, password hash
        db/
          base.py         DeclarativeBase, TimestampMixin
          session.py      async_sessionmaker, get_db dependency
        models/          one file per table (user.py, member.py ...)
        schemas/         one file per resource (request + response)
        services/        business logic, one file per domain
        tasks/           scheduler.py, expiry_job.py, reminder_job.py
      alembic/
        versions/        migration scripts

```

tests/	
unit/	service layer tests (no HTTP)
integration/	httpx AsyncClient tests
main.py	FastAPI app factory
requirements.txt	
Dockerfile	
frontend/	
src/	
api/	axios instance + one file per resource
components/	shared UI (Button, Modal, Table, Form...)
pages/	one folder per route
auth/	
members/	
subscriptions/	
classes/	
reports/	
hooks/	useAuth, useReservations, useClasses...
store/	auth context (access token in memory)
package.json	
Dockerfile	
docker-compose.yml	postgres + backend + frontend + redis
.env.example	all required environment variables

Appendix A: Glossary

Term	Definition
Member	A registered user with the 'member' role and an associated member profile record.
Administrator	A user with full system management rights; the 'admin' role.
Trainer	A fitness center staff member with the 'trainer' role.
Subscription	A time-bounded access contract linking a member to a subscription type.
Subscription Type	A template defining a name, duration (days), and base fee. Examples: Monthly, Quarterly, Annual.
Personalized Subscription	A subscription where the total price includes a base fee plus a per-session charge for personal training.
Reservation	A booking of one participant spot in a specific class.
Waitlist	An ordered queue of members waiting for a spot in a full class.
Facility	A named physical room or area in the fitness center that can host classes.
Class	A scheduled group training session led by a trainer in a facility with a defined capacity.
JWT	JSON Web Token -- a signed token encoding user identity and role claims, used for stateless authentication.
RBAC	Role-Based Access Control -- permission model where access is determined by the user's role.
ORM	Object-Relational Mapper -- SQLAlchemy in this project; maps Python classes to database tables.
Alembic	Database migration tool for SQLAlchemy; manages schema version history.

Term	Definition
REQ-N	Unique identifier for a functional requirement, where N is a sequential number.
RON	Romanian Leu -- the currency used for all monetary values in this system.
Bani	1/100 of a RON. All monetary values are stored as integers in bani to avoid floating-point issues.
GDPR	General Data Protection Regulation (EU 2016/679) -- EU law governing personal data handling.
PCI-DSS	Payment Card Industry Data Security Standard -- security requirements for systems handling card payments.
WCAG 2.1	Web Content Accessibility Guidelines -- standards for making web content accessible to users with disabilities.
TLS	Transport Layer Security -- cryptographic protocol securing all HTTP communications (HTTPS).
SPA	Single-Page Application -- a web app where navigation occurs without full page reloads (React in this project).
RTO	Recovery Time Objective -- maximum acceptable time to restore service after a failure.
RPO	Recovery Point Objective -- maximum acceptable amount of data loss measured in time.

Appendix B: Analysis Models

The following analysis models will be produced during the detailed design phase and attached to this appendix as separate documents:

- **Entity-Relationship Diagram:** Complete ER diagram for all 10 database tables, showing cardinalities and foreign key relationships.
- **Class Diagram:** UML class diagram for the backend service layer, showing the domain model with Member, Subscription, Class, Reservation, Trainer, and Facility entities.
- **Sequence Diagrams:** Detailed interaction flows for: (1) user login, (2) online payment processing including webhook handling, (3) class reservation with waitlist promotion, and (4) automatic subscription expiry job.
- **State Diagrams:** State transition diagrams for Subscription status (active, expired, cancelled, restricted) and Reservation status (active, cancelled, waitlisted).
- **Data Flow Diagram:** Level-1 DFD showing data flows between the frontend, backend, database, payment gateway, and notification services.

Appendix C: To Be Determined List

#	TBD Item	Section	Owner	Target Date
TBD-1	Final selection between Stripe and PayU as the payment gateway. Depends on fee comparison and local bank compatibility.	3.3, 4.22	Project Manager	TBD
TBD-2	Deployment infrastructure decision (cloud provider, managed DB vs self-hosted). Affects Section 2.4 and Section 7.	2.4, 7.1	CTO / Lead Dev	TBD
TBD-3	Whether SMS notifications (Twilio) will be included in v1.0 or deferred. Affects 4.38 and 4.39.	4.38, 4.39	Project Manager	TBD
TBD-4	Exact late-cancellation policy enforcement (fee vs. flag only). Currently only flagged per REQ in 4.34.	4.34, 5.5	Stakeholder	TBD
TBD-5	Whether a native mobile app (iOS/Android) is planned for v1.1, to inform API design decisions.	2.1	Stakeholder	TBD